
The Abstract Interpretation Framework

Lecture 4 — Static Analysis by Abstract Interpretation

David Pichardie

Fall 2026

The collecting semantics computes sets of reachable states:

$$\llbracket c \rrbracket : \mathcal{P}(\Sigma) \longrightarrow \mathcal{P}(\Sigma).$$

Lecture 3 gave us:

- lattices and least fixpoints;
- a terminating solver when the ACC holds;
- the abstract-interpreter pattern.

Today's question

What must an abstract computation satisfy to approximate the collecting semantics soundly?

Approximation with signs

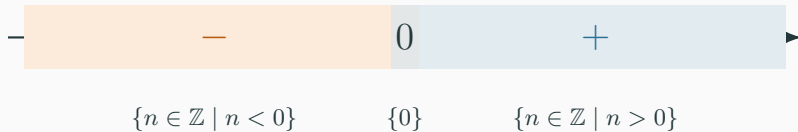
A γ -only framework: soundness of operators

Sound operations on signs

§ 1

Approximation with signs

Forget values, retain signs



$$\mathbb{S} \triangleq \{\perp, -, 0, +, \top\}.$$

Concretization gives meaning

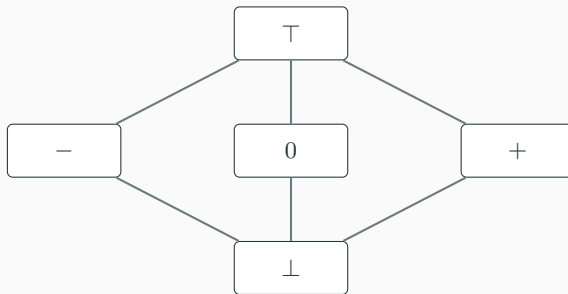
Each abstract symbol represents a numerical property, i.e. an element of $\mathcal{P}(\mathbb{Z})$; the concretization function $\gamma_V : \mathbb{S} \rightarrow \mathcal{P}(\mathbb{Z})$ gives this meaning.

a	$\gamma_V(a)$	property
$\perp$	$\emptyset$	impossible
$-$	$\{n \mid n < 0\}$	negative
0	$\{0\}$	zero
$+$	$\{n \mid n > 0\}$	positive
$\top$	$\mathbb{Z}$	unknown

The precision order on signs

The partial order represents precision:

$a \sqsubseteq b$ means that a is more precise than b .



Lower means more precise; moving upward loses information.

Order means loss of information

$$a \sqsubseteq b \iff \gamma_V(a) \subseteq \gamma_V(b).$$

$$\begin{array}{ccccc} \perp & \sqsubseteq & + & \sqsubseteq & \top \\ \downarrow \gamma_V & & \downarrow \gamma_V & & \downarrow \gamma_V \\ \emptyset & \subseteq & \{n \mid n > 0\} & \subseteq & \mathbb{Z} \end{array}$$

Going upward remains sound and becomes less precise.

Any numerical property can be approximated

A numerical property $S \subseteq \mathbb{Z}$ is over-approximated by $a \in \mathbb{S}$ iff

$$S \subseteq \gamma_V(a).$$

numerical property S	most precise sound abstraction
$\emptyset$	$\perp$
$\{0\}$	0
$\{n \in \mathbb{Z} \mid n > 3\}$	$+$
$\{n \in \mathbb{Z} \mid n \leq 0\}$	$\top$
$\{n \in \mathbb{Z} \mid n \neq 0\}$	$\top$

At worst, $\top$ is always a sound abstraction.

The sign order is a lattice

Every pair has a least upper bound and a greatest lower bound.

Join

$a \sqcup b = \text{least upper bound of } a, b$
 $- \sqcup 0 = \top.$

Meet

$a \sqcap b = \text{greatest lower bound of } a, b$
 $- \sqcap 0 = \perp.$

It is finite, hence a complete lattice satisfying the ACC.

Remarks

- Join may lose precision: $\gamma_V(a) \cup \gamma_V(b) \subseteq \gamma_V(a \sqcup b)$. Why?
- Meet is exact here: $\gamma_V(a \sqcap b) = \gamma_V(a) \cap \gamma_V(b)$.

$$\sigma^\# \in \Sigma^\# \triangleq \text{Var} \rightarrow \mathbb{S}.$$

Each abstract state represents a property of concrete states:

$$\gamma_\Sigma : \Sigma^\# \longrightarrow \mathcal{P}(\Sigma).$$

$$\gamma_\Sigma(\sigma^\#) \triangleq \{\sigma \in \Sigma \mid \forall x, \sigma(x) \in \gamma_V(\sigma^\#(x))\}.$$

Example 1

$$[x \mapsto +, y \mapsto -]$$

represents the concrete states satisfying $x > 0 \wedge y < 0$.

Order and lattice on abstract states

The order is pointwise:

$$\sigma_1^\# \sqsubseteq^\# \sigma_2^\# \iff \forall x, \sigma_1^\#(x) \sqsubseteq \sigma_2^\#(x).$$

So are joins, meets, and bounds:

$$(\sigma_1^\# \sqcup^\# \sigma_2^\#)(x) \triangleq \sigma_1^\#(x) \sqcup \sigma_2^\#(x),$$

$$(\sigma_1^\# \sqcap^\# \sigma_2^\#)(x) \triangleq \sigma_1^\#(x) \sqcap \sigma_2^\#(x).$$

$$\perp^\#(x) \triangleq \perp, \quad \top^\#(x) \triangleq \top.$$

Completeness and ACC

The pointwise operations make $\Sigma^\#$ a complete lattice. Since Var and $\mathbb{S}$ are finite, it also satisfies the ACC.

$$\sigma_1^\# \sqsubseteq^\# \sigma_2^\# \implies \gamma_\Sigma(\sigma_1^\#) \subseteq \gamma_\Sigma(\sigma_2^\#).$$

Exercise — properties of γ_Σ

For abstract values, we had

$$\gamma_V(a) \subseteq \gamma_V(b) \implies a \sqsubseteq b.$$

1. Show that the corresponding implication is false on abstract states:

$$\gamma_\Sigma(\sigma_1^\#) \subseteq \gamma_\Sigma(\sigma_2^\#) \not\Rightarrow \sigma_1^\# \sqsubseteq^\# \sigma_2^\#.$$

2. Show that γ_Σ is exact for intersection:

$$\gamma_\Sigma(\sigma_1^\# \sqcap^\# \sigma_2^\#) = \gamma_\Sigma(\sigma_1^\#) \cap \gamma_\Sigma(\sigma_2^\#).$$

Exercise — properties of γ_Σ

For abstract values, we had

$$\gamma_V(a) \subseteq \gamma_V(b) \implies a \sqsubseteq b.$$

1. Show that the corresponding implication is false on abstract states:

$$\gamma_\Sigma(\sigma_1^\#) \subseteq \gamma_\Sigma(\sigma_2^\#) \not\Rightarrow \sigma_1^\# \sqsubseteq^\# \sigma_2^\#.$$

2. Show that γ_Σ is exact for intersection:

$$\gamma_\Sigma(\sigma_1^\# \sqcap^\# \sigma_2^\#) = \gamma_\Sigma(\sigma_1^\#) \cap \gamma_\Sigma(\sigma_2^\#).$$

Solution

1. Take $\text{Var} = \{x, y\}$ and $\sigma_1^\# = [x \mapsto \perp, y \mapsto +]$, $\sigma_2^\# = [x \mapsto \perp, y \mapsto -]$. Both concretizations are empty, but $+$ $\not\sqsubseteq$ $-$.

Exercise — properties of γ_Σ

For abstract values, we had

$$\gamma_V(a) \subseteq \gamma_V(b) \implies a \sqsubseteq b.$$

1. Show that the corresponding implication is false on abstract states:

$$\gamma_\Sigma(\sigma_1^\#) \subseteq \gamma_\Sigma(\sigma_2^\#) \not\Rightarrow \sigma_1^\# \sqsubseteq^\# \sigma_2^\#.$$

2. Show that γ_Σ is exact for intersection:

$$\gamma_\Sigma(\sigma_1^\# \sqcap^\# \sigma_2^\#) = \gamma_\Sigma(\sigma_1^\#) \cap \gamma_\Sigma(\sigma_2^\#).$$

Solution

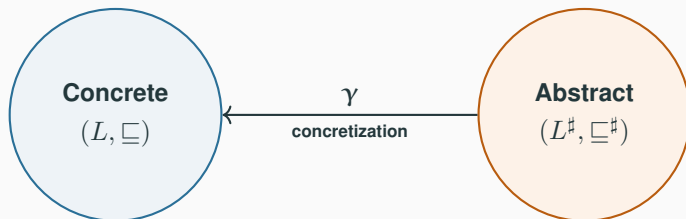
2. Write $A = \sigma_1^\#$ and $B = \sigma_2^\#$. For every concrete state σ ,

$$\begin{aligned} \sigma \in \gamma_\Sigma(A \sqcap^\# B) &\iff \forall x, \sigma(x) \in \gamma_V(A(x) \sqcap B(x)) \\ &\iff \forall x, \sigma(x) \in \gamma_V(A(x)) \cap \gamma_V(B(x)) \\ &\iff (\forall x, \sigma(x) \in \gamma_V(A(x))) \wedge (\forall x, \sigma(x) \in \gamma_V(B(x))) \\ &\iff \sigma \in \gamma_\Sigma(A) \cap \gamma_\Sigma(B). \end{aligned}$$

§2

A γ -only framework: soundness of operators

The framework



We assume only a monotone concretization:

$$\gamma : L^\# \longrightarrow L, \quad a \subseteq^\# b \implies \gamma(a) \subseteq \gamma(b).$$

For now, this generic picture has two instances:

	L	$L^\#$	γ
values	$\mathcal{P}(\mathbb{Z})$	$\mathbb{S}$	γ_V
states	$\mathcal{P}(\Sigma)$	$\Sigma^\#$	γ_Σ

Sound abstract transformers — assignment

Start with assignment. Its collecting transformer is

$$\llbracket x := e \rrbracket S = \{ \sigma[x \mapsto \llbracket e \rrbracket \sigma] \mid \sigma \in S \}.$$

For an abstract input $\sigma^\sharp$, execute every state it represents:

$$\llbracket x := e \rrbracket (\gamma_\Sigma(\sigma^\sharp)) = \{ \sigma[x \mapsto \llbracket e \rrbracket \sigma] \mid \sigma \in \gamma_\Sigma(\sigma^\sharp) \}.$$

The abstract result must describe *all* these successors:

$$\boxed{\llbracket x := e \rrbracket (\gamma_\Sigma(\sigma^\sharp)) \subseteq \gamma_\Sigma(\text{assign}^\sharp(x, e, \sigma^\sharp)).}$$

Sound abstract transformers — general form

In general, for $f : L \rightarrow L$ and $f^\sharp : L^\sharp \rightarrow L^\sharp$:

$$f(\gamma(a)) \sqsubseteq \gamma(f^\sharp(a)).$$

The order on L induces the pointwise order on functions $h, k : L^\sharp \rightarrow L$:

$$h \sqsubseteq k \iff \forall a \in L^\sharp, h(a) \sqsubseteq k(a).$$

The same condition can then be written

$$f \circ \gamma \sqsubseteq \gamma \circ f^\sharp.$$

More generally, for $f : L_1 \times \dots \times L_n \rightarrow L$ and $f^\sharp : L_1^\sharp \times \dots \times L_n^\sharp \rightarrow L^\sharp$:

$$f(\gamma_1(a_1), \dots, \gamma_n(a_n)) \sqsubseteq \gamma(f^\sharp(a_1, \dots, a_n)).$$

Exercise — joins and unions

Let L and $L^\sharp$ be lattices, and let $\gamma : L^\sharp \rightarrow L$ be a monotone **concretization** function.

Prove that abstract join soundly represents concrete join.

Exercise — joins and unions

Let L and $L^\sharp$ be lattices, and let $\gamma : L^\sharp \rightarrow L$ be a monotone **concretization** function.

Prove that abstract join soundly represents concrete join.

Solution

Since $a \sqsubseteq^\sharp a \sqcup^\sharp b$ and $b \sqsubseteq^\sharp a \sqcup^\sharp b$, monotonicity gives

$$\gamma(a) \sqsubseteq \gamma(a \sqcup^\sharp b), \quad \gamma(b) \sqsubseteq \gamma(a \sqcup^\sharp b).$$

Hence $\gamma(a \sqcup^\sharp b)$ is an upper bound of both concretizations, so

$$\gamma(a) \sqcup \gamma(b) \sqsubseteq \gamma(a \sqcup^\sharp b).$$

A join may introduce additional imprecision

Assume that x is initially arbitrary. The guard refines the `then` branch to $x > 0$.

Exact merge

```
if x > 0 then skip
  else x := 1
```

$$+ \sqcup + = +$$

$$\gamma_V(+) \cup \gamma_V(+) = \gamma_V(+ \sqcup +).$$

No additional information is lost at the merge:
 $x > 0$.

Inexact merge

```
if x > 0 then skip
  else x := 0
```

$$+ \sqcup 0 = \top$$

$$\gamma_V(+) \cup \gamma_V(0) \subsetneq \gamma_V(+ \sqcup 0) = \mathbb{Z}.$$

The merge introduces spurious states with $x < 0$.

Consequence

After the second program, the analysis loses $x \geq 0$ and may report a false alarm for a later check of $x < 0$.

Meet is not automatically sound

Since $a \sqcap^\# b \sqsubseteq^\# a, b$, monotonicity gives

$$\gamma(a \sqcap^\# b) \sqsubseteq \gamma(a) \sqcap \gamma(b).$$

This is the *opposite* of the inequality required to soundly approximate concrete meet. Soundness therefore requires

$$\gamma(a) \sqcap \gamma(b) \sqsubseteq \gamma(a \sqcap^\# b),$$

and hence equality.

Signs preserve meet

$$\gamma_V(a \sqcap b) = \gamma_V(a) \sqcap \gamma_V(b).$$

Meet is therefore exact, and in particular sound, for signs.

Exercise — monotonicity is not enough

Show that monotonicity of the concretization does not guarantee that abstract meet soundly approximates concrete meet.

Hint

Take $L = \mathcal{P}(\mathbb{Z})$. It is enough to consider another abstraction of signs (four points are enough).

Exercise — monotonicity is not enough

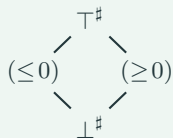
Show that monotonicity of the concretization does not guarantee that abstract meet soundly approximates concrete meet.

Hint

Take $L = \mathcal{P}(\mathbb{Z})$. It is enough to consider another abstraction of signs (four points are enough).

Solution

Take the diamond lattice $L^\# = \{\perp^\#, (\leq 0), (\geq 0), \top^\#\}$ and define $\gamma : L^\# \rightarrow \mathcal{P}(\mathbb{Z})$ as follows:



$$\gamma(\perp^\#) = \emptyset, \quad \gamma(\leq 0) = \{n \in \mathbb{Z} \mid n \leq 0\},$$

$$\gamma(\top^\#) = \mathbb{Z}, \quad \gamma(\geq 0) = \{n \in \mathbb{Z} \mid n \geq 0\}.$$

This concretization is monotone. Yet $(\leq 0) \sqcap^\# (\geq 0) = \perp^\#$, while

$$\gamma(\perp^\#) = \emptyset \quad \text{and} \quad \gamma(\leq 0) \cap \gamma(\geq 0) = \{0\}.$$

Thus abstract meet loses the concrete value 0 and is not sound.

§3

Sound operations on signs

Already established

Bounds have the expected meaning:

$$\gamma_V(\perp) = \emptyset, \quad \gamma_V(\top) = \mathbb{Z}.$$

Join is sound for union:

$$\gamma_V(a) \cup \gamma_V(b) \subseteq \gamma_V(a \sqcup b).$$

Meet is exact for intersection:

$$\gamma_V(a \sqcap b) = \gamma_V(a) \cap \gamma_V(b).$$

Then

These local operations can be assembled into a sound abstract interpreter for complete commands.

Still to define and prove sound

1. arithmetic operations, starting with $+$ [#];
2. evaluation of expressions;
3. assignment on abstract states;
4. filters for boolean conditions.

$+^\sharp$	$\perp$	$-$	0	$+$	$\top$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
$-$	$\perp$	$-$	$-$	$\top$	$\top$
0	$\perp$	$-$	0	$+$	$\top$
$+$	$\perp$	$\top$	$+$	$+$	$\top$
$\top$	$\perp$	$\top$	$\top$	$\top$	$\top$

$$\gamma_V(a) + \gamma_V(b) \subseteq \gamma_V(a +^\sharp b), \quad \text{with} \quad A + B \triangleq \{x + y \mid x \in A, y \in B\}.$$

Returning $\top$ every time would be sound, but not precise.

Concrete set addition

Writing $\{z < 0\}$ for $\{z \in \mathbb{Z} \mid z < 0\}$:

- $\emptyset + A = \emptyset$;
- $\{z = 0\} + \{z > 0\} = \{z > 0\}$;
- $\{z < 0\} + \{z > 0\} = \mathbb{Z}$;
- $\{z > 0\} + \{z > 0\} = \{z > 1\}$.

Exercise — abstract subtraction

Complete the most precise sound table. The $\perp$ row and column are given.

$-^\#$	$\perp$	$-$	0	$+$	$\top$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
$-$	$\perp$				
0	$\perp$				
$+$	$\perp$				
$\top$	$\perp$				

Every entry must satisfy

$$\gamma_V(a) - \gamma_V(b) \subseteq \gamma_V(a -^\# b), \quad \text{with} \quad A - B \triangleq \{x - y \mid x \in A, y \in B\}.$$

Exercise — abstract subtraction

Complete the most precise sound table. The $\perp$ row and column are given.

$-^\#$	$\perp$	$-$	0	$+$	$\top$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
$-$	$\perp$	$\top$	$-$	$-$	$\top$
0	$\perp$	$+$	0	$-$	$\top$
$+$	$\perp$	$+$	$+$	$\top$	$\top$
$\top$	$\perp$	$\top$	$\top$	$\top$	$\top$

Every entry must satisfy

$$\gamma_V(a) - \gamma_V(b) \subseteq \gamma_V(a -^\# b), \quad \text{with} \quad A - B \triangleq \{x - y \mid x \in A, y \in B\}.$$

Exercise — abstract multiplication

Complete the most precise sound table. The $\perp$ row and column are given.

$\times^\#$	$\perp$	$-$	0	$+$	$\top$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
$-$	$\perp$				
0	$\perp$				
$+$	$\perp$				
$\top$	$\perp$				

Every entry must satisfy

$$\gamma_V(a) \times \gamma_V(b) \subseteq \gamma_V(a \times^\# b), \quad \text{with} \quad A \times B \triangleq \{xy \mid x \in A, y \in B\}.$$

Exercise — abstract multiplication

Complete the most precise sound table. The $\perp$ row and column are given.

$\times^\#$	$\perp$	$-$	0	$+$	$\top$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
$-$	$\perp$	$+$	0	$-$	$\top$
0	$\perp$	0	0	0	0
$+$	$\perp$	$-$	0	$+$	$\top$
$\top$	$\perp$	$\top$	0	$\top$	$\top$

Every entry must satisfy

$$\gamma_V(a) \times \gamma_V(b) \subseteq \gamma_V(a \times^\# b), \quad \text{with} \quad A \times B \triangleq \{xy \mid x \in A, y \in B\}.$$

$$n^\sharp \triangleq \begin{cases} - & n < 0, \\ 0 & n = 0, \\ + & n > 0. \end{cases}$$

$$\text{eval}^\sharp(n, \sigma^\sharp) \triangleq n^\sharp,$$

$$\text{eval}^\sharp(x, \sigma^\sharp) \triangleq \sigma^\sharp(x),$$

$$\text{eval}^\sharp(a_1 + a_2, \sigma^\sharp) \triangleq \text{eval}^\sharp(a_1, \sigma^\sharp) +^\sharp \text{eval}^\sharp(a_2, \sigma^\sharp),$$

$$\text{eval}^\sharp(a_1 - a_2, \sigma^\sharp) \triangleq \text{eval}^\sharp(a_1, \sigma^\sharp) -^\sharp \text{eval}^\sharp(a_2, \sigma^\sharp),$$

$$\text{eval}^\sharp(a_1 \times a_2, \sigma^\sharp) \triangleq \text{eval}^\sharp(a_1, \sigma^\sharp) \times^\sharp \text{eval}^\sharp(a_2, \sigma^\sharp).$$

Theorem 2 (Soundness of expression evaluation)

If $\sigma \in \gamma_\Sigma(\sigma^\sharp)$, then

$$\llbracket a \rrbracket \sigma \in \gamma_V(\text{eval}^\sharp(a, \sigma^\sharp)).$$

Proof: structural induction on a .

Sound operators compose

The soundness proof for $\text{eval}^\sharp$ implicitly composed sound functions at each syntax node.

Let $g : L \rightarrow M$ and $f : M \rightarrow N$ be monotone, with sound abstract counterparts $g^\sharp$ and $f^\sharp$:

$$g \circ \gamma_L \sqsubseteq \gamma_M \circ g^\sharp, \quad f \circ \gamma_M \sqsubseteq \gamma_N \circ f^\sharp.$$

Then composition is sound:

$(f \circ g) \circ \gamma_L = f \circ (g \circ \gamma_L)$	by associativity
$\sqsubseteq f \circ (\gamma_M \circ g^\sharp)$	by soundness of $g^\sharp$ and monotonicity of f
$\sqsubseteq \gamma_N \circ f^\sharp \circ g^\sharp$	by soundness of $f^\sharp$
$= \gamma_N \circ (f^\sharp \circ g^\sharp)$	by associativity.

$$\text{assign}^\#(x, a, \sigma^\#) \triangleq \sigma^\#[x \mapsto \text{eval}^\#(a, \sigma^\#)].$$

Expression soundness gives

$$\llbracket x := a \rrbracket(\gamma_\Sigma(\sigma^\#)) \subseteq \gamma_\Sigma(\text{assign}^\#(x, a, \sigma^\#)).$$

Proof. By definition,

$$\llbracket x := a \rrbracket S = \{\sigma[x \mapsto \llbracket a \rrbracket \sigma] \mid \sigma \in S\}, \quad \gamma_\Sigma(\sigma^\#) = \{\sigma \mid \forall y, \sigma(y) \in \gamma_V(\sigma^\#(y))\}.$$

Take $\sigma' \in \llbracket x := a \rrbracket(\gamma_\Sigma(\sigma^\#))$. Then some $\sigma \in \gamma_\Sigma(\sigma^\#)$ satisfies $\sigma' = \sigma[x \mapsto \llbracket a \rrbracket \sigma]$. For every variable y :

- if $y = x$, expression soundness gives $\sigma'(x) \in \gamma_V(\text{eval}^\#(a, \sigma^\#))$;
- if $y \neq x$, then $\sigma'(y) = \sigma(y) \in \gamma_V(\sigma^\#(y))$.

Hence $\sigma' \in \gamma_\Sigma(\text{assign}^\#(x, a, \sigma^\#))$.

Example 3

$$[x \mapsto 0] \xrightarrow{x := x + 1} [x \mapsto +].$$

Soundness is expressed compactly, using the pointwise order on functions:

$$\llbracket b \rrbracket \circ \gamma_\Sigma \sqsubseteq \gamma_\Sigma \circ \llbracket b \rrbracket^\#.$$

Equivalently:

$$\forall \sigma \in \gamma_\Sigma(\sigma^\#), \quad \llbracket b \rrbracket \sigma = \texttt{tt} \implies \sigma \in \gamma_\Sigma(\llbracket b \rrbracket^\# \sigma^\#).$$

Indeed, the concrete filter is

$$\llbracket b \rrbracket S \triangleq \{ \sigma \in S \mid \llbracket b \rrbracket \sigma = \texttt{tt} \}.$$

input and condition	$\longrightarrow$	output
$x \mapsto \top, \quad x > 0$	$\longrightarrow$	$x \mapsto +$
$x \mapsto \top, \quad x \leq 0$	$\longrightarrow$	$x \mapsto \top$
$x \mapsto -, \quad x = 0$	$\longrightarrow$	$\perp$

Implementing filters on signs

For guards comparing one variable with zero, first define

$$\text{refine}(x, s, \sigma^\sharp) \triangleq \sigma^\sharp[x \mapsto \sigma^\sharp(x) \sqcap s].$$

Since meet is sound for intersection (and in fact exact) on signs,

$$\llbracket x < 0 \rrbracket^\sharp \sigma^\sharp = \text{refine}(x, -, \sigma^\sharp), \quad \llbracket x = 0 \rrbracket^\sharp \sigma^\sharp = \text{refine}(x, 0, \sigma^\sharp), \quad \llbracket x > 0 \rrbracket^\sharp \sigma^\sharp = \text{refine}(x, +, \sigma^\sharp).$$

What remains

The guards $x < 0$, $x = 0$, and $x > 0$ correspond to individual signs. For each other comparison, the abstract filter is defined by considering each possible current sign separately.

Exercise — derive the remaining atomic filters

For an atomic guard $x \bowtie 0$, let $F_{\bowtie}^{\#}(s)$ be the refined value of x when its current sign is s . The first three columns follow from the preceding slide. Complete the remaining three columns.

s	$x < 0$	$x = 0$	$x > 0$	$x \leq 0$	$x \geq 0$	$x \neq 0$
$\perp$	$\perp$	$\perp$	$\perp$			
$-$	$-$	$\perp$	$\perp$			
0	$\perp$	0	$\perp$			
$+$	$\perp$	$\perp$	$+$			
$\top$	$-$	0	$+$			

The atomic filter handles only this syntactic form:

$$\text{atom}^{\#}(p, A) \triangleq \begin{cases} A[x \mapsto F_{\bowtie}^{\#}(A(x))] & p = (x \bowtie 0), \\ A & \text{otherwise (sound but imprecise).} \end{cases}$$

Exercise — derive the remaining atomic filters

For an atomic guard $x \bowtie 0$, let $F_{\bowtie}^{\sharp}(s)$ be the refined value of x when its current sign is s . The first three columns follow from the preceding slide. Complete the remaining three columns.

s	$x < 0$	$x = 0$	$x > 0$	$x \leq 0$	$x \geq 0$	$x \neq 0$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$
$-$	$-$	$\perp$	$\perp$	$-$	$\perp$	$-$
0	$\perp$	0	$\perp$	0	0	$\perp$
$+$	$\perp$	$\perp$	$+$	$\perp$	$+$	$+$
$\top$	$-$	0	$+$	$\top$	$\top$	$\top$

The atomic filter handles only this syntactic form:

$$\text{atom}^{\sharp}(p, A) \triangleq \begin{cases} A[x \mapsto F_{\bowtie}^{\sharp}(A(x))] & p = (x \bowtie 0), \\ A & \text{otherwise (sound but imprecise).} \end{cases}$$

Solution

Intersect $\gamma_V(s)$ with the integers satisfying the guard, then take the most precise representable sign. For example, non-positive integers are not represented by one element, so $F_{\leq}^{\sharp}(\top) = \top$.

The recursive sign filter

For a normalized condition b , let $\text{atom}^\#$ be the transformer from the previous slide and define

$$\llbracket b \rrbracket^\# A \triangleq \text{filter}^\#(b, A) \triangleq \begin{cases} A & b = \text{true}, \\ \perp^\# & b = \text{false}, \\ \text{atom}^\#(p, A) & b = p, \\ \text{filter}^\#(b_2, \text{filter}^\#(b_1, A)) & b = b_1 \wedge b_2, \\ \text{filter}^\#(b_1, A) \sqcup^\# \text{filter}^\#(b_2, A) & b = b_1 \vee b_2. \end{cases}$$

Why is there no $\neg b$ case?

Normalize boolean expressions before filtering: push negations inward with De Morgan's laws, then negate comparison operators, for example

$$\neg(b_1 \wedge b_2) \rightsquigarrow \neg b_1 \vee \neg b_2, \quad \neg(x < 0) \rightsquigarrow x \geq 0.$$

Exercise — beyond atomic sign filters

The current implementation returns A when the guard is not of the form $x \bowtie 0$. Can we return a more precise sound sign state?

1. $\llbracket 3 \times x < y \rrbracket^\# [x \mapsto \top, y \mapsto -] = ?$
2. $\llbracket x - y > 1 \rrbracket^\# [x \mapsto +, y \mapsto \top] = ?$

Looking ahead

The general case requires backward propagation through arithmetic expressions. This more advanced implementation will be studied with the interval abstraction in Project 2 and Lecture 5.

Exercise — beyond atomic sign filters

The current implementation returns A when the guard is not of the form $x \bowtie 0$. Can we return a more precise sound sign state?

1. $\llbracket 3 \times x < y \rrbracket^\sharp [x \mapsto \top, y \mapsto -] = ?$
2. $\llbracket x - y > 1 \rrbracket^\sharp [x \mapsto +, y \mapsto \top] = ?$

Solution

$$\llbracket 3 \times x < y \rrbracket^\sharp [x \mapsto \top, y \mapsto -] = [x \mapsto -, y \mapsto -].$$

Indeed, $3x < y < 0$ implies $x < 0$.

$$\llbracket x - y > 1 \rrbracket^\sharp [x \mapsto +, y \mapsto \top] = [x \mapsto +, y \mapsto \top].$$

Every sign of y is compatible with some positive x , so no non-relational refinement is possible.

Looking ahead

The general case requires backward propagation through arithmetic expressions. This more advanced implementation will be studied with the interval abstraction in Project 2 and Lecture 5.

Exercise — soundness of the sign filter

Let p range over atomic comparisons and b over normalized boolean conditions.

1. State the soundness theorem for $\text{atom}^\sharp$.
2. Assuming this theorem, prove the soundness of $\text{filter}^\sharp(b, \cdot)$.

Exercise — soundness of the sign filter

Let p range over atomic comparisons and b over normalized boolean conditions.

1. State the soundness theorem for $\text{atom}^\sharp$.
2. Assuming this theorem, prove the soundness of $\text{filter}^\sharp(b, \cdot)$.

Solution

Answer to 1. For every atomic comparison p ,

$$\llbracket p \rrbracket \circ \gamma_\Sigma \sqsubseteq \gamma_\Sigma \circ \text{atom}^\sharp(p, \cdot).$$

Equivalently, for every abstract state A ,

$$\llbracket p \rrbracket(\gamma_\Sigma(A)) \subseteq \gamma_\Sigma(\text{atom}^\sharp(p, A)).$$

We admit its proof, which is a tedious case analysis.

Solution — base cases and conjunction

Write $F_b^\sharp \triangleq \text{filter}^\sharp(b, \cdot)$. We prove $\llbracket b \rrbracket \circ \gamma_\Sigma \sqsubseteq \gamma_\Sigma \circ F_b^\sharp$ by induction on b .

Base cases. For every abstract state A :

$$\llbracket \text{true} \rrbracket(\gamma_\Sigma(A)) = \gamma_\Sigma(A) = \gamma_\Sigma(F_{\text{true}}^\sharp(A)),$$

since $F_{\text{true}}^\sharp(A) = A$;

$$\llbracket \text{false} \rrbracket(\gamma_\Sigma(A)) = \emptyset = \gamma_\Sigma(F_{\text{false}}^\sharp(A)),$$

since $F_{\text{false}}^\sharp(A) = \perp^\sharp$;

$$\llbracket p \rrbracket(\gamma_\Sigma(A)) \subseteq \gamma_\Sigma(\text{atom}^\sharp(p, A)) = \gamma_\Sigma(F_p^\sharp(A)),$$

by the admitted theorem.

Conjunction. For every A ,

$$\begin{aligned} \llbracket b_1 \wedge b_2 \rrbracket(\gamma_\Sigma(A)) &= \llbracket b_2 \rrbracket(\llbracket b_1 \rrbracket(\gamma_\Sigma(A))) \\ &\subseteq \llbracket b_2 \rrbracket(\gamma_\Sigma(F_{b_1}^\sharp(A))) \end{aligned}$$

by definition of $\llbracket b_1 \wedge b_2 \rrbracket$

by IH for b_1 and monotonicity of $\llbracket b_2 \rrbracket$

$$\begin{aligned} &\subseteq \gamma_\Sigma(F_{b_2}^\sharp(F_{b_1}^\sharp(A))) \\ &= \gamma_\Sigma(F_{b_1 \wedge b_2}^\sharp(A)) \end{aligned}$$

by IH for b_2

by definition of $F_{b_1 \wedge b_2}^\sharp$.

Disjunction. For every A ,

$$\begin{aligned}\llbracket b_1 \vee b_2 \rrbracket(\gamma_\Sigma(A)) &= \llbracket b_1 \rrbracket(\gamma_\Sigma(A)) \cup \llbracket b_2 \rrbracket(\gamma_\Sigma(A)) \\ &\subseteq \gamma_\Sigma(F_{b_1}^\#(A)) \cup \gamma_\Sigma(F_{b_2}^\#(A)) \\ &\subseteq \gamma_\Sigma(F_{b_1}^\#(A) \sqcup^\# F_{b_2}^\#(A)) \\ &= \gamma_\Sigma(F_{b_1 \vee b_2}^\#(A))\end{aligned}$$

by definition of $\vee$

by both IHs

by soundness of join

by definition of $F_{b_1 \vee b_2}^\#$.

Lecture 4 — what we have established

What we now have

- the sign lattice and its concretization;
- a precise statement of operator soundness;
- sound arithmetic and expression evaluation;
- sound assignment and boolean filters.

What remains

- assemble these operators into an abstract semantics for commands;
- handle `while` by relating the concrete and abstract least fixed points;
- conclude soundness for every command;
- then ask how to obtain the *best* sound abstractions.

Next lecture

We first complete the soundness proof for the abstract interpreter, then introduce intervals and Galois connections to address precision.